Pulsebook: a Limit-Order-Book Matching Engine Validated on Real Binance Flow

Mahesh Thakur

October 2026

Overview

Pulsebook is a central limit order book (CLOB) matching engine in Go, implementing the price-time priority model used by every major exchange: orders rest at price levels in arrival order, incoming orders cross at the best available prices, and trades print at the resting order's price. The engine is validated two ways: an invariant fuzzer (20,000 random operations with full book-integrity checks after every operation) and a replay of real Binance BTCUSDT market flow - 100,000 actual aggressor trades from the public data archive - through the matching path.

Why fixed-point arithmetic

A floating-point price of $82,000.52 cannot be represented exactly in binary; 0.1 + 0.2 != 0.3 is not a curiosity in a matching engine, it is a mis-crossed book. Pulsebook uses int64 ticks (0.01 USDT) and int64 lots (1e-6 BTC) everywhere. Comparison is exact, aggregation is exact, and the invariant checker can assert strict inequalities that floats would make fragile.

Architecture

```
AddLimit(buy, px, qty):
  while best ask <= px:        # crossing
      fill FIFO at best ask    # price-time priority
      print trade at ask price # maker price
  rest remainder in bid level  # FIFO append

Book state:
  bids: sorted []int64 prices (desc) -> level{FIFO queue of *Order}
  asks: sorted []int64 prices (asc) -> level{...}
  orders: map[id]*Order        # O(1) cancel via element pointer
```

- Insert: O(log L) level search + amortized O(L) slice insert, L = active levels
- Match: O(1) per fill (level head), O(L) level retirement
- Cancel: O(1)
- Depth snapshot: O(L)

Correctness: two independent validations

Invariant fuzzer

20,000 pseudo-random operations (45% resting limits, 15% cancels, 40% marketable

takers). After EVERY operation: book uncrossed (best bid < best ask), level lists sorted, FIFO queues time-ordered, no zero-quantity orders. Result: 16,896 orders, 16,045 trades, zero violations.

Real Binance flow replay

The public data.binance.vision aggTrades dump for BTCUSDT on 2026-10-19 - 689,545 real aggressor trades; the benchmark replays the first 100,000. Each real trade becomes a market order (isBuyerMaker=true means the aggressor sold). Between aggressors, a market-maker policy keeps ~1 BTC of liquidity in the top 3 levels near the touch and rebuilds a side's ladder if it empties (the one-sided-book death spiral was found and fixed during benchmarking: once a side emptied, no refill logic ran at all and every subsequent aggressor silently matched nothing).

Result: 100,000 real aggressor trades and ~190,000 total book events in 0.16s (1.03M events/s), 164,339 executions, invariants held at the end. Trade prices track the real day's tape around $82,000-82,350.

Measured performance

Method: single goroutine, sequential calls, in-process engine API (no HTTP), fixed seed, linux/amd64, Go 1.26.6. Reproduce: `go run ./cmd/bench`.

Operation	Measured
Limit insert (resting)	429 ns/op - 2.33M orders/s
Cancel	162 ns/op - 6.16M cancels/s
Matching (crossing order)	1,589 ns/op - 0.63M orders/s
Real Binance replay	1.03M events/s, 164,339 executions, invariants held

For calibration: production exchange gateways quote sub-100ns matching in C++ with kernel-bypass networking; a garbage-collected engine at ~430ns insert / ~1.6us match is in the expected band and honest about it.

Testing

Ten tests under `go test -race`: non-crossing orders never trade, full cross prints at maker price, partial fills rest the remainder, FIFO within a level, price beats time across levels, book walking in price order, sell-side crossing, cancel semantics (including double-cancel), IOC market orders never rest, and the 20k-op invariant fuzz.

Limitations

- Single instrument, single goroutine (production shards by symbol)
- Slice-based levels; a red-black tree or arena allocator is the next lever for very deep books
- The replay validates matching semantics; it is not a backtester (no queue position or latency model)
- Sequential benchmarks by design

Conclusion

The interesting part of a matching engine is not the hot loop - it is the contract: price-time priority, maker-priced trades, and a book that is never crossed. Pulsebook states

that contract as executable invariants, checks them after every one of 20,000 fuzzed operations and after 100,000 real market events, and publishes the measured cost of keeping it.